
Valdris: El Núcleo Profundo

Memoria y Documento de Diseño

Grupo H12GEXTRA · Estructuras de Datos · Metodología de la Programación

Grupo	H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez
Versión	v1.0 — Entrega final
Lenguaje	Java 21 · JavaFX · Gson 2.10.1 · IntelliJ IDEA
Fecha	28 de mayo de 2026

1. Introducción

Valdris: El Núcleo Profundo es un juego de exploración y combate por turnos desarrollado en Java 21 con interfaz gráfica JavaFX. El jugador se adentra en una red de habitaciones conectadas mediante un grafo, recorre zonas con identidad propia, combate enemigos, recoge objetos, resuelve puzzles y avanza hasta el enfrentamiento final contra el Parásito del Núcleo.

El proyecto combina diseño orientado a objetos, estructuras de datos propias, persistencia en JSON, pruebas unitarias y una interfaz gráfica funcional. El objetivo principal no ha sido únicamente crear un juego jugable, sino construir un sistema software organizado, justificable y coherente con los requisitos de las asignaturas de Estructuras de Datos y Metodología de la Programación.

Durante la partida, el usuario elige uno de los tres personajes disponibles: Kael, Syra o Dorath. Cada personaje tiene estadísticas, estilo de combate y afinidades diferentes. A partir de esa elección, el jugador progresa por cinco zonas principales de Valdris, cada una con enemigos, salas, recompensas, puzzles y elementos narrativos propios.

El juego utiliza un sistema de turnos dividido en fases. En cada turno, el jugador puede realizar movimiento, recogida/interacción, uso de objetos y ataque, respetando las restricciones indicadas en el enunciado. Después actúan los enemigos de la sala. La partida termina cuando el jugador alcanza y resuelve el desenlace final, o cuando se cumple una condición de derrota, como quedarse sin vida o agotar los turnos disponibles.

2. Alcance del Documento

Este documento unifica la memoria final y el documento de diseño del proyecto. Su objetivo es presentar de forma ordenada el sistema desarrollado, explicar las decisiones tomadas y justificar cómo se cumplen los requisitos del enunciado.

La memoria recoge:

- La descripción general del juego.
- Los requisitos funcionales y no funcionales.
- El diseño del modelo de dominio.
- La arquitectura orientada a objetos.
- Los casos de uso principales.
- Las estructuras de datos utilizadas y sus costes.
- Los algoritmos principales.
- La persistencia mediante JSON.
- La interfaz gráfica JavaFX.
- Los diagramas UML obligatorios.
- Los contratos e invariantes del sistema.
- La estrategia de pruebas.
- El uso de inteligencia artificial durante el desarrollo.
- Las decisiones de diseño más relevantes.
- La crítica del proyecto y posibles mejoras futuras.

Las guías internas del proyecto incluidas en la carpeta `docs/` se han utilizado como base de diseño y planificación. Este documento consolida esa información y la actualiza al estado final del proyecto implementado.

3. Requisitos del Sistema

Esta sección recoge los requisitos principales que guían el diseño y la implementación de *Valdris: El Núcleo Profundo*. Los requisitos parten del enunciado de la práctica y se concretan en las decisiones de diseño propias del grupo.

3.1 Requisitos Funcionales

El mapa se modela como un grafo de salas mediante `Dungeon` y `Grafo<Room, String>`.

Cada `Room` contiene una matriz de celdas `Cell[][]` con dimensiones propias.

La interfaz JavaFX representa la sala como una rejilla visual con jugador,

enemigos, objetos y elementos interactivos.

El jugador se mueve con BFS sobre celdas transitables y solo puede elegir destinos válidos.

Las puertas y escaleras conectan salas del grafo mediante accesos configurados.

El sistema calcula distancia y ruta global hacia el progreso principal usando búsqueda sobre el grafo y BFS interno.

Existen armas, armaduras, pociones, accesorios y objetos narrativos con efectos propios.

Los objetos pueden aparecer en celdas del suelo, cofres, drops de enemigos o recompensas narrativas.

El jugador mantiene inventario, equipo activo y objetos narrativos separados.

La interfaz permite equipar objetos y consumir pociones durante la fase correspondiente del turno.

El combate se resuelve con ataque, defensa, rango, línea de visión, efectos de estado y aleatoriedad controlada.

La IA enemiga decide ataques, movimientos, efectos, invocaciones o espera según tipo de enemigo.

La victoria se produce al superar el enfrentamiento final y cerrar el desenlace narrativo.

El jugador pierde si muere, agota turnos globales o se cumple una derrota por límite de sala.

`LectorJSON` serializa el estado de partida en `partida_valdris.json`.

El juego reconstruye el estado guardado a partir del JSON de partida.

`DungeonConfigLoader` carga la configuración base desde `config/configuracion_inicial_valdris.json`.

El juego mantiene un log estructurado de acciones, combate, movimiento, objetos, puzzles y eventos.

Al terminar, el jugador puede ver el resumen de partida y exportarlo a

`resumen_valdris.json`.

3.2 Requisitos No Funcionales

Todo el proyecto está desarrollado en Java 21.

La capa `Valdris.ui` contiene pantallas, vistas, controlador y modelo JavaFX.

El proyecto usa estructuras propias como `ListaSimplementeEnlazada`, `Cola`, `Pila`, árboles y `Grafo`.

El código se organiza en modelo, lógica, persistencia, excepciones e interfaz.

Las clases principales usan atributos privados y métodos públicos controlados.

Se usan excepciones personalizadas como `InvalidMoveException`, `InvalidAttackException` y `GameStateException`.

Las capas no visuales están cubiertas con tests JUnit 5.

Los ficheros JSON usan DTOs planos y referencias por identificador para evitar ciclos.

`TASKS.md`, `COMMIT_LOG.md`, JSON de ejemplo, UML, imágenes y esta memoria documentan el desarrollo.

3.3 Requisitos Específicos del Enunciado

Código fuente entregable: cumplido mediante el proyecto Maven y la carpeta `src/`.

Repositorio y ZIP: pendiente únicamente del empaquetado final de entrega.

Documento de diseño: cubierto por esta memoria consolidada.

Memoria PDF: esta memoria se exportará a PDF al cierre del proyecto.

UML obligatorio: parcialmente cubierto en la carpeta `diagramas/`; faltan diagramas específicos que se completarán antes de la exportación final.

JSON de ejemplo: cubierto con `config/configuracion_inicial_valdris.json`, `partida_valdris.json` y `resumen_valdris.json`.

Pruebas: cubierto por la carpeta `tests/` y la suite JUnit registrada en

`TASKS.md`.

Diario de IA: cubierto por `COMMIT_LOG.md`.

Bocetos de interfaz: cubierto como base visual por la carpeta `imagenes/`, que se referencia en la sección de interfaz.

Vídeo explicativo: preparado por el grupo como entregable externo al repositorio.

3.4 Alcance Final Implementado

El alcance final del proyecto incluye una partida completa con selección de personaje, introducción narrativa, exploración de cinco zonas, salas normales y secretas, cofres, armas, objetos, enemigos con IA, mini-bosses, puzzles, guardado y carga JSON, combate final, desenlace narrativo y resumen final visible/exportable.

Quedan fuera del alcance implementado algunas ampliaciones opcionales del enunciado, como deshacer acciones mediante pila, costes variables de terreno con Dijkstra en la lógica principal o multijugador. Estas opciones no eran necesarias para cumplir el núcleo obligatorio del proyecto.

4. Diseño General del Juego

4.1 Concepto General

Valdris: El Núcleo Profundo plantea una exploración por habitaciones en la que el jugador avanza desde la superficie devastada de Valdris hasta el núcleo donde se concentra la amenaza final. La estructura del juego combina recorrido, gestión de recursos, combate táctico, decisiones de equipamiento y resolución de eventos narrativos.

La partida no se basa en recorrer un mapa lineal simple. El mundo se organiza como una red de salas conectadas por puertas, escaleras y pasadizos. Cada sala

tiene una matriz propia de celdas, de modo que el jugador debe resolver tanto el movimiento local dentro de la habitación como la progresión global entre nodos del grafo.

4.2 Flujo Principal de Partida

El flujo principal de una partida nueva es:

- El jugador abre el menú principal.
- Selecciona iniciar una nueva partida.
- Lee la introducción narrativa del mundo de Valdris.
- Elige entre Kael, Syra o Dorath.
- Entra en la pantalla de descenso hacia el Núcleo Profundo.
- Comienza la partida en la sala inicial.
- Explora salas, combate enemigos y recoge objetos.
- Activa accesos, resuelve puzzles y descubre salas secretas.
- Avanza por las cinco zonas del mapa.
- Llega al tramo final y se enfrenta al Parásito del Núcleo.
- Obtiene una victoria narrativa o una derrota.
- Consulta el resumen final de la partida.

El flujo también permite cargar una partida guardada desde el menú principal. En ese caso, el sistema reconstruye el estado guardado en JSON y devuelve al jugador a la sala, posición, inventario, turnos y situación correspondiente.

4.3 Zonas del Mundo

El mapa se divide en cinco zonas principales:

Funciona como primera zona de exploración y aprendizaje. Introduce el combate básico, las primeras recompensas y el tono decadente de Valdris.

Introduce una ambientación natural corrompida y enemigos más centrados en movilidad, distancia y control del espacio.

Presenta espacios más cerrados, enemigos resistentes y una progresión más exigente de combate y recursos.

Refuerza la presencia mágica del mundo, los puzzles y las amenazas relacionadas con efectos de estado.

Actúa como cierre de la partida. Contiene el tramo de no retorno, la presencia de Malachar y el combate final contra el Parásito.

Cada zona tiene un tono visual diferenciado en la interfaz. Las salas aplican ligeros tintes de color y los pasillos de transición muestran degradados entre zonas para hacer visible el avance por Valdris.

4.4 Personajes Jugables

El jugador puede elegir uno de tres personajes. La elección afecta al estilo de partida, estadísticas iniciales, arma inicial y afinidad con determinadas armas.

Kael es el personaje más resistente y está orientado al combate cuerpo a cuerpo. Su estilo favorece acercarse a los enemigos y aguantar mejor los golpes.

Syra destaca por su movilidad y alcance medio. Su diseño favorece reposición, ataques a distancia y control de la posición dentro de la sala.

Dorath tiene menor movilidad, pero mayor alcance mágico. Su estilo se basa en mantener distancia y aprovechar armas o efectos relacionados con magia.

4.5 Sistema de Turnos

El turno del jugador se divide en fases ordenadas:

- Movimiento.
- Recogida o interacción.
- Uso de objeto.
- Ataque.
- Turno de enemigos.

El jugador puede realizar como máximo una acción de cada tipo por turno. Siempre se resuelve primero el movimiento y después las acciones. Al terminar las fases del jugador, actúan los enemigos vivos de la sala actual.

Si el jugador cambia de sala mediante una puerta, escalera o acceso equivalente, el proceso de turno se reinicia en la nueva habitación. Esto evita que el jugador entre en una sala nueva en una fase avanzada y mantiene la lectura del turno clara.

4.6 Victoria y Derrota

La victoria se produce al superar el combate final contra el Parásito del Núcleo y cerrar el desenlace narrativo correspondiente al personaje elegido. El final mantiene una resolución de sacrificio: el protagonista y Malachar mueren por el último ataque del Parásito, pero Valdris sobrevive.

La derrota puede producirse por varias causas:

- El jugador pierde todos sus puntos de vida.
- Se agotan los turnos globales disponibles.
- Se agota el límite de turnos de una sala con temporizador.
- El jugador muere por daño directo, efectos de estado, puzzles u otros eventos.

El sistema registra la causa concreta de derrota para mostrarla al jugador en la pantalla final y en el resumen de partida.

4.7 Diseño Narrativo y Progresión

La progresión del juego combina hitos fijos y variación controlada. El mapa base, las salas, los accesos importantes, los mini-bosses, cofres secretos y objetivos principales están definidos por la configuración inicial. A la vez, algunos elementos como posiciones de enemigos normales, drops, pools de objetos y orden de puzzles conservan variación para que la partida no sea completamente predecible.

Esta decisión permite cumplir el requisito de configuración inicial desde JSON sin eliminar la parte lúdica de incertidumbre y rejugabilidad. El JSON define el mundo y sus reglas de generación, mientras que la lógica del juego aplica esa configuración para construir una partida concreta.

5. Modelo de Dominio y Diseño Orientado a Objetos

5.1 Organización General del Dominio

El modelo de dominio se ha dividido en entidades pequeñas y con responsabilidades claras. La intención principal ha sido separar el estado del juego de la lógica

que lo modifica y de la interfaz que lo muestra al usuario.

Las entidades principales son:

Representado por **Player**. Contiene el personaje elegido, estadísticas, posición actual, inventario, equipo, efectos activos y estado de vida.

Unit actúa como clase base para cualquier entidad con vida, ataque, defensa, rango, movimiento y efectos de estado. De ella derivan **Player**, **Enemy** y aliados o enemigos especiales.

Enemy representa a los enemigos normales. Las variantes especiales se modelan mediante tipos, estadísticas y clases concretas cuando necesitan comportamiento propio, como **MiniBossEnemy**, **ParasitoEnemy** y **MalacharAlly**.

Item es la clase abstracta base de todos los objetos. Sus especializaciones principales son **Weapon**, **Armor**, **Potion**, **Accessory** y **NarrativeItem**.

Dungeon representa el mundo completo como un grafo de habitaciones. Cada **Room** representa una sala y contiene una matriz **Cell[][]**. Cada **Cell** describe una casilla concreta del tablero.

Container y **Chest** representan cofres y recompensas. **HiddenPassage** modela pasos secretos que pueden activarse desde casillas especiales.

GameState, **GameConfig**, **LoadedGame**, **GameSummary** y **LectorJSON** agrupan los datos necesarios para guardar, cargar, configurar y resumir una partida.

La capa JavaFX está formada por vistas, controlador y modelo de interfaz. Esta capa no contiene la lógica principal del juego, sino que invoca operaciones de modelo y representa visualmente los cambios.

5.2 Separación por Capas

El proyecto mantiene una separación por capas para evitar dependencias circulares y facilitar las pruebas:

Define los tipos básicos del dominio: **CellType**, **EffectType**, **ItemType**, **CharacterType**, **EnemyType**, **MiniBossType**, **Phase**, **Effect** e ítems.

Define personajes, enemigos y entidades con vida. Usa las clases base de

ítems, efectos y enumerados.

Define salas, celdas, dungeon, contenedores y pasadizos. Puede contener unidades e ítems, pero no depende de JavaFX.

Contiene movimiento, combate, visión, IA, turnos, generación y puzzles. Opera sobre el modelo, pero no conoce la interfaz gráfica.

Transforma el estado del juego a JSON y reconstruye partidas o configuraciones.

Usa DTOs para evitar referencias circulares entre objetos del dominio.

Contiene JavaFX. Incluye `MainApp`, `GameModel`, `GameController` y las vistas visuales. Esta capa puede usar el resto, pero ninguna capa inferior depende de ella.

5.3 Herencia y Polimorfismo

La herencia se usa en los puntos donde existe una relación clara de especialización:

`Unit` concentra atributos comunes como vida, ataque, defensa, movimiento, rango y efectos. `Player`, `Enemy`, `MiniBossEnemy`, `ParasitoEnemy` y `MalacharAlly` reutilizan esa base y añaden datos o comportamientos específicos.

`Item` define la identidad común de un objeto: identificador, nombre, tipo, rareza o descripción. `Weapon`, `Armor`, `Potion`, `Accessory` y

`NarrativeItem` concretan el efecto que tiene cada tipo dentro de la partida.

`Container` define el comportamiento común de un contenedor interactuable y

`Chest` representa el cofre concreto que aparece en las salas.

El polimorfismo permite tratar elementos distintos mediante una referencia base cuando se necesita una operación general. Por ejemplo, el inventario almacena objetos como `Item`, mientras que la lógica de uso o equipamiento distingue la especialización concreta cuando corresponde.

5.4 Encapsulamiento

El estado interno de las clases del dominio se mantiene privado. El acceso se

realiza mediante métodos públicos controlados, lo que evita modificaciones directas desde otras capas.

Esta decisión es importante en varios puntos:

- La vida de una unidad no debe cambiarse sin validar muerte o límites.
- La posición del jugador debe respetar el movimiento permitido.
- Los efectos de estado deben añadirse, procesarse y caducar de forma ordenada.
- El inventario debe conservar coherencia entre objetos almacenados y equipo.
- Las celdas deben reflejar correctamente paredes, puertas, objetos y enemigos.

El encapsulamiento también facilita la persistencia, porque el estado que se exporta a JSON se obtiene de forma controlada y se reconstruye mediante métodos específicos.

5.5 Identidad de las Entidades

Muchas entidades usan identificadores estables. Esto es especialmente relevante en salas, objetos, enemigos, cofres y configuración JSON.

Ejemplos de identificadores usados en el diseño:

- Salas como **S1-A**, **S2-SEC** o **S5-D**.
- Armas como **W1**, **W5** o **W12**.
- Objetos narrativos y accesorios como **AC1**.
- Cofres y disparadores secretos mediante identificadores de configuración.

Estos identificadores permiten guardar partidas sin serializar referencias complejas entre objetos. El JSON puede indicar que una entidad pertenece a una sala o celda concreta, y la carga reconstruye la relación a partir del identificador.

5.6 Relación con los Diagramas UML

Los diagramas de clases incluidos en la carpeta **diagramas/** representan esta estructura general del dominio. En ellos deben verse las relaciones principales:

- **Unit** como base de jugador, enemigos y entidades especiales.
- **Item** como base de armas, armaduras, pociones, accesorios y objetos narrativos.

-
- `Dungeon`, `Room` y `Cell` como núcleo del mapa.
 - Las clases de lógica operando sobre el modelo sin formar parte directa del estado persistente.
 - La capa JavaFX separada del modelo y conectada a través de `GameModel` y `GameController`.

Esta separación permite que el diseño sea explicable en UML y que las pruebas unitarias se centren en las capas no visuales.

6. Estructuras de Datos y Costes

6.1 Criterio General

El proyecto evita las estructuras de datos estándar de `java.util` en la lógica principal evaluada. En su lugar, utiliza las estructuras propias incluidas en el repositorio:

- `ListaSimplementeEnlazada<T>`.
- `ListaCircular<T>`.
- `Cola<T>`.
- `Pila<T>`.
- `Grafo<DN, DA>`.

El objetivo no es solo cumplir el enunciado, sino justificar por qué cada estructura encaja con el problema que resuelve.

6.2 Lista Simplemente Enlazada

La `ListaSimplementeEnlazada<T>` se usa para colecciones donde interesa recorrer elementos en orden y añadir o eliminar datos sin depender de arrays dinámicos de la biblioteca estándar.

Usos principales:

- Inventario del jugador.
- Objetos narrativos.
- Enemigos de una sala.
- Efectos activos de unidades.

-
- Log de eventos.
 - Listas de resultados, acciones o candidatos de generación.

Costes principales:

- Insertar al inicio: $O(1)$.
- Insertar al final: $O(n)$ si no existe puntero final.
- Buscar por posición o condición: $O(n)$.
- Recorrer la lista completa: $O(n)$.
- Eliminar un elemento conocido por búsqueda: $O(n)$.

La lista encaja bien porque la mayoría de operaciones del juego recorren colecciones pequeñas o medianas: enemigos de una sala, objetos del inventario o efectos activos. El coste lineal es aceptable y mantiene una implementación sencilla y explicable.

6.3 Lista Circular

La `ListaCircular<T>` forma parte de las estructuras propias implementadas en el proyecto base. No es la estructura principal para el inventario o el mapa, pero se conserva y se prueba como parte de la entrega de estructuras de datos.

Su utilidad natural dentro del dominio sería recorrer secuencias cíclicas, por ejemplo patrones repetitivos de turnos, rotaciones de efectos o ciclos de eventos. En la versión final del juego esas necesidades se resuelven con lógica directa, por lo que la lista circular queda como estructura implementada, validada y disponible para ampliaciones.

Costes habituales:

- Añadir en una posición conocida: $O(1)$.
- Buscar un elemento concreto: $O(n)$.
- Recorrer todos los elementos: $O(n)$.
- Avanzar al siguiente elemento desde un nodo conocido: $O(1)$.

6.4 Cola

La `Cola<T>` se utiliza cuando el orden de procesamiento debe ser primero en entrar, primero en salir.

Usos principales:

- Búsqueda BFS de movimiento dentro de una sala.
- Exploración de vecinos en algoritmos de camino.
- Procesamiento ordenado de nodos pendientes.

Costes principales:

- Encolar: $O(1)$.
- Desencolar: $O(1)$.
- Consultar si está vacía: $O(1)$.

La cola es la estructura natural para BFS. Permite explorar primero las celdas o nodos a menor distancia antes de avanzar a distancias mayores. Por eso se usa en **BFSMovimiento** y en la lógica de rutas.

6.5 Grafo

El **Grafo** $\langle DN, DA \rangle$ representa el mapa global. Cada nodo es una habitación y cada arista representa una conexión entre salas.

Usos principales:

- Conexión entre habitaciones del dungeon.
- Cálculo de rutas entre salas.
- Representación de pasillos, puertas, escaleras y transiciones.
- Control del punto de no retorno hacia el tramo final.

Costes aproximados:

- Insertar nodo: depende de la implementación interna, normalmente $O(n)$ si se comprueba duplicado por recorrido.
- Insertar arista: $O(n)$ o $O(\text{grado})$ según búsqueda de nodos y adyacencias.
- Consultar vecinos de una sala: $O(\text{grado})$.
- BFS sobre el grafo: $O(V + E)$, siendo V salas y E conexiones.

El grafo es adecuado porque Valdris no es una lista lineal de salas. Existen ramas, salas secretas, pasillos de transición y conexiones especiales. El grafo permite razonar sobre distancia global y progreso sin mezclarlo con la matriz interna de cada habitación.

6.6 Matriz de Celdas

Cada sala se representa con una matriz `Cell[][]`. Esta estructura no sustituye al grafo, sino que resuelve un nivel distinto del problema: el espacio local dentro de una habitación.

Usos principales:

- Paredes.
- Suelo transitable.
- Puertas y puertas ocultas.
- Escaleras.
- Objetos en el suelo.
- Cofres y palancas.
- Posiciones de jugador y enemigos.
- Celdas especiales de puzzle o activación.

Costes principales:

- Acceso a una celda por fila y columna: $O(1)$.
- Comprobar vecinos en cuatro direcciones: $O(1)$.
- Recorrer toda una sala: $O(\text{filas} * \text{columnas})$.
- BFS interno: $O(\text{filas} * \text{columnas})$ en el peor caso.

La matriz es la estructura más adecuada para una sala por celdas, ya que permite validar movimientos y representar obstáculos de forma directa.

6.7 Árbol de Decisión

La IA enemiga usa un árbol de decisión para elegir acciones. Cada nodo evalúa una condición y conduce a otra comprobación o a una acción final.

Usos principales:

- Decidir si un enemigo puede atacar.
- Decidir si debe moverse hacia el jugador.
- Decidir comportamiento especial de guardianes, destructores o invocadores.
- Mantener una IA explicable y separada del combate.

Costes principales:

- Evaluar una decisión: $O(h)$, siendo h la altura del árbol.
- En el árbol usado, h es pequeña y constante para la partida.

El árbol de decisión encaja porque las reglas de IA son jerárquicas. Primero se comprueban condiciones importantes, como poder atacar, y después se decide entre moverse, usar una habilidad especial o esperar.

6.8 Pila

La `Pila<T>` forma parte de las estructuras disponibles del proyecto. En el alcance final, el juego no implementa una función completa de deshacer acciones, por lo que la pila no es una estructura central de la partida.

Su uso queda documentado como estructura disponible y como mejora posible para una versión futura. Sería adecuada para almacenar acciones previas del jugador y permitir revertir movimientos o decisiones si el enunciado opcional de deshacer se quisiera ampliar.

Costes principales:

- Apilar: $O(1)$.
- Desapilar: $O(1)$.
- Consultar cima: $O(1)$.

6.9 Justificación de Costes en el Juego

El tamaño del juego permite que los costes elegidos sean adecuados. Las salas tienen dimensiones controladas, el número de enemigos por sala es limitado y el grafo contiene un número fijo de habitaciones.

Por ello:

- BFS sobre una sala es eficiente y fácil de justificar.
- BFS sobre el grafo es suficiente para rutas globales.
- Recorrer listas enlazadas no supone un coste problemático.
- La matriz permite acceso directo a celdas concretas.
- El árbol de decisión mantiene constante el coste de IA por enemigo.

La combinación de grafo global y matriz local es una de las decisiones más importantes del diseño. El grafo responde a la pregunta de qué salas están conectadas, mientras que `Cell[][]` responde a cómo se puede mover el jugador dentro de cada sala.

7. Algoritmos Principales

7.1 Movimiento con BFS

El movimiento dentro de una sala se calcula mediante búsqueda en anchura. La clase `BFSMovimiento` recibe la sala actual, la posición inicial y el límite de movimiento del jugador o enemigo.

El algoritmo explora las celdas vecinas en orden creciente de distancia. Para ello usa una cola de posiciones pendientes y una estructura auxiliar para marcar las celdas ya visitadas.

La búsqueda solo considera válidas las celdas transitables:

- Suelo normal.
- Puertas accesibles.
- Escaleras.
- Celdas especiales activadas.
- Celdas sin pared ni bloqueo.

No se permite atravesar paredes, cofres cerrados, obstáculos o celdas ocupadas por unidades cuando la regla concreta lo impide.

Coste:

- En el peor caso se revisa toda la sala.
- Si la sala tiene `f` filas y `c` columnas, el coste es $O(f * c)$.

Este coste es adecuado porque las salas tienen tamaño limitado y el algoritmo se ejecuta sobre una sola habitación, no sobre todo el mapa a la vez.

7.2 Camino entre Habitaciones

El recorrido global entre salas se resuelve sobre el grafo del dungeon. La clase `BFSCaminoMinimo` permite calcular rutas entre nodos del mapa.

Cada sala actúa como nodo y cada puerta, pasillo o escalera como conexión. El algoritmo busca la ruta con menor número de conexiones, lo que permite orientar

al jugador hacia el progreso principal o comprobar accesibilidad entre zonas.

Coste:

- $O(V + E)$, siendo V el número de salas y E el número de conexiones.

La elección de BFS es suficiente porque las conexiones del mapa no usan pesos de coste variable en la lógica principal. Todas las transiciones entre salas se tratan como pasos equivalentes.

7.3 Línea de Visión

La clase `LineaDeVision` comprueba si una unidad puede atacar a otra a distancia sin que haya paredes u obstáculos entre medias.

El algoritmo recorre el segmento entre atacante y objetivo y valida las celdas intermedias. Si encuentra una pared o bloqueo de visión, el ataque no puede realizarse aunque el objetivo esté dentro del rango.

La línea de visión es especialmente importante para:

- Arqueros.
- Francotiradores.
- Magos.
- Dorath.
- Enemigos destructores o controladores.

Coste:

- $O(d)$, siendo d la distancia en celdas entre atacante y objetivo.

El coste es bajo porque el rango de ataque máximo está acotado por las estadísticas de unidades y armas.

7.4 Combate

La clase `CombatManager` centraliza la resolución de ataques. El combate tiene en cuenta ataque total, arma equipada, defensa total, penetración, rango, efectos de estado y aleatoriedad de daño.

La fórmula general de daño es:

```
danio = max(0, ataqueTotal * aleatorio - defensaEfectiva)
```

La defensa efectiva se calcula restando la penetración del arma a la defensa del objetivo. El resultado nunca puede ser negativo.

El combate también controla:

- Fallos por ceguera.
- Daño adicional por maldición.
- Restricciones por parálisis.
- Aplicación de efectos especiales.
- Muerte de unidades.
- Drops y recompensas cuando corresponde.

Coste:

- La resolución de un ataque individual es $O(1)$.
- Si se revisan efectos activos, el coste pasa a $O(e)$, siendo e el número de efectos de la unidad.

Como el número de efectos activos es pequeño, el coste real se mantiene bajo y predecible.

7.5 IA de Enemigos

La IA enemiga combina reglas específicas por tipo de enemigo con un árbol de decisión. Las clases `IAEnemigo`, `MiniBossAI` y `ArbolDecisionIA` separan la decisión de la resolución del combate.

El proceso general es:

- Comprobar si el enemigo está vivo y puede actuar.
- Comprobar si el jugador está dentro del rango de ataque.
- Validar línea de visión cuando el ataque lo requiere.
- Elegir ataque, habilidad especial, movimiento o espera.
- Ejecutar la acción elegida y registrar el resultado.

Cada tipo de enemigo tiene una identidad jugable:

- **Warrior**: avanza hacia el jugador y ataca cuerpo a cuerpo.
- **Berserker**: prioriza presión y movimiento agresivo.
- **Guardian**: protege una zona concreta.

-
- **Archer**: mantiene distancia útil.
 - **Sniper**: ataca con mayor alcance y control de enfriamiento.
 - **Destructor**: aplica daño de área.
 - **Controlador**: aplica efectos de estado.
 - **Invocador**: crea enemigos adicionales de forma limitada.

Coste:

- Decisión del árbol: $O(h)$.
- Movimiento con BFS si necesita desplazarse: $O(f * c)$.
- Ataque directo: $O(1)$ o $O(e)$ si procesa efectos.

El coste más alto se produce cuando un enemigo necesita calcular movimiento, pero solo se hace sobre la sala actual.

7.6 Puzzles y Activadores

La clase **PuzzleManager** gestiona interacciones especiales asociadas a salas, palancas, secuencias y disparadores.

Los puzzles se apoyan en la configuración inicial y en el estado de la sala. De esta forma se puede mantener una base determinista del mundo sin eliminar variación interna en patrones o recompensas cuando el diseño lo permite.

Los activadores se usan para:

- Abrir puertas ocultas.
- Cambiar el estado de celdas especiales.
- Desbloquear salas secretas.
- Registrar eventos narrativos o de progreso.

Coste:

- Activar un disparador concreto suele ser $O(1)$.
- Si se recorren celdas de una sala para cambiar estados, el coste es $O(f * c)$.

Este diseño evita codificar casos especiales directamente en la interfaz. La lógica de puzzle queda en la capa correspondiente.

7.7 Gestión de Turnos

TurnManager controla el avance por fases y valida qué acciones puede realizar el jugador en cada momento.

Las fases principales son:

- Movimiento.
- Recogida o interacción.
- Uso de objeto.
- Ataque.
- Turno de enemigos.

El gestor impide repetir una acción ya consumida y reinicia la fase cuando el jugador cambia de sala. También controla turnos globales, límites de sala y procesamiento de efectos.

Coste:

- Cambiar de fase es $O(1)$.
- Procesar efectos del jugador es $O(e)$.
- Procesar enemigos depende del número de enemigos vivos de la sala.

Si hay n enemigos, el turno enemigo tiene coste aproximado $O(n * a)$, donde a es el coste de decidir y ejecutar la acción de cada enemigo.

7.8 Generación Controlada del Mundo

DungeonGenerator y **DungeonConfigLoader** trabajan juntos para construir la partida inicial. La configuración JSON define la estructura del mundo, mientras que la generación aplica reglas de variación controlada.

La configuración fija:

- Habitaciones.
- Dimensiones.
- Celdas especiales.
- Puertas.
- Paredes interiores.
- Cofres.
- Conexiones.
- Salas secretas.
- Posición inicial.

-
- Objetivo principal.

La generación puede variar:

- Posiciones válidas de enemigos normales.
- Drops.
- Algunos patrones de puzzle.
- Recompensas aleatorias no críticas.

Con esta división, la partida cumple el requisito de configuración inicial desde JSON sin convertirse en una experiencia completamente rígida.

8. Persistencia JSON

8.1 Objetivo de la Persistencia

La persistencia permite guardar, cargar y revisar el estado de una partida. En Valdris se usan ficheros JSON porque son legibles, fáciles de revisar y adecuados para representar objetos compuestos mediante DTOs.

La persistencia cubre tres necesidades:

- Configuración inicial del mundo.
- Estado guardado de una partida.
- Resumen final exportable.

La biblioteca usada para serializar y deserializar es Gson.

8.2 Configuración Inicial

La configuración inicial se encuentra en:

```
config/configuracion_inicial_valdris.json
```

Este fichero define la base determinista del mundo. No guarda una partida en curso, sino la estructura desde la que se construye una partida nueva.

Incluye información como:

- Identificador de cada sala.
- Zona a la que pertenece.

-
- Dimensiones.
 - Matriz de celdas relevantes.
 - Posición inicial del jugador.
 - Objetivo principal.
 - Conexiones entre salas.
 - Puertas y escaleras.
 - Puertas ocultas.
 - Triggers secretos.
 - Cofres y opciones de recompensa.
 - Enemigos fijos o candidatos de aparición.
 - Puzzles y eventos narrativos.

`DungeonConfigLoader` lee este fichero y construye un `GameConfig`. Después, `DungeonGenerator` transforma esa configuración en objetos reales del dominio: `Dungeon`, `Room`, `Cell`, enemigos, cofres y elementos interactivos.

8.3 Estado de Partida

El estado de partida se guarda en:

```
partida_valdris.json
```

Este fichero representa la situación concreta de una partida en un momento dado.

Su objetivo es permitir continuar desde ese punto.

El estado guardado incluye:

- Personaje elegido.
- Vida actual.
- Posición del jugador.
- Sala actual.
- Turno y fase.
- Inventario.
- Equipo activo.
- Objetos narrativos obtenidos.
- Enemigos vivos y muertos.
- Cofres abiertos.
- Celdas modificadas.
- Puertas ocultas descubiertas.

-
- Efectos activos.
 - Log de eventos.
 - Condiciones de progreso.

La clase `LectorJSON` coordina el guardado y la carga. La clase `GameState` contiene los DTOs usados para serializar el estado sin depender directamente de referencias complejas entre objetos.

8.4 Resumen Final

El resumen final se exporta en:

```
resumen_valdris.json
```

Este fichero no sirve para continuar una partida, sino para documentar el resultado de la sesión.

Incluye datos como:

- Resultado final.
- Personaje utilizado.
- Causa de victoria o derrota.
- Turnos consumidos.
- Enemigos derrotados.
- Objetos recogidos.
- Salas visitadas.
- Eventos relevantes.
- Resumen narrativo.

En la interfaz final, el jugador puede ver el resumen antes de exportarlo. Esto evita que el JSON sea el único lugar donde consultar el resultado.

8.5 Uso de DTOs

La persistencia utiliza DTOs para separar el modelo interno del formato JSON. Un DTO contiene datos simples y serializables, pero no lógica de juego.

Esta decisión evita problemas como:

- Referencias circulares entre dungeon, salas y celdas.
- Serialización accidental de objetos JavaFX.

-
- Guardado de datos calculables o temporales.
 - Dependencia directa entre el JSON y la implementación interna exacta.

Por ejemplo, una celda no necesita guardar una referencia completa a su sala. Es suficiente con guardar su fila, columna, tipo y estado relevante. De forma similar, una conexión entre salas puede guardarse usando identificadores.

8.6 Reconstrucción al Cargar

La carga de una partida no se limita a leer un objeto plano. El sistema debe reconstruir relaciones entre entidades.

El proceso general es:

- Leer el JSON con Gson.
- Validar que los datos mínimos existen.
- Cargar la configuración base del mundo si es necesario.
- Reconstruir el dungeon y sus salas.
- Aplicar cambios guardados sobre celdas, cofres y enemigos.
- Reconstruir jugador, inventario, equipo y efectos.
- Restaurar turnos, fase, sala actual y posición.
- Devolver un objeto `LoadedGame` listo para la interfaz.

Esta reconstrucción separada es necesaria porque el estado guardado contiene identificadores y DTOs, mientras que el juego necesita objetos conectados y operativos.

8.7 Ventajas del Formato JSON

El uso de JSON aporta varias ventajas al proyecto:

- Permite revisar manualmente la configuración inicial.
- Facilita comprobar partidas guardadas durante depuración.
- Hace posible incluir ejemplos de entrega.
- Evita formatos binarios difíciles de explicar.
- Encaja con Gson, que es una dependencia externa permitida.

Además, al existir JSON de configuración, partida y resumen, el proyecto muestra tres usos distintos de persistencia: entrada de configuración, estado de juego y salida de resultados.

8.8 Limitaciones y Control de Errores

La persistencia depende de que los identificadores del JSON sean coherentes. Si una sala, objeto o conexión apunta a un identificador inexistente, la carga no puede reconstruir correctamente la partida.

Para reducir ese riesgo:

- Se usan identificadores estables.
- La configuración principal se mantiene en un único fichero.
- La lógica de carga centraliza la reconstrucción.
- Los errores se tratan mediante excepciones de estado cuando corresponde.
- Los JSON de ejemplo se conservan en el repositorio para revisión.

Esta aproximación mantiene el sistema comprensible y facilita detectar errores durante la fase final de entrega.

9. Interfaz Gráfica JavaFX

9.1 Objetivo de la Interfaz

La interfaz JavaFX permite jugar una partida completa sin depender de consola.

Su objetivo es mostrar de forma clara el estado del juego, las acciones disponibles y la información narrativa necesaria.

La interfaz no sustituye a la lógica del juego. Las vistas muestran datos y recogen acciones del usuario, pero las reglas de movimiento, combate, persistencia y turnos se resuelven en las capas inferiores.

9.2 Estructura General

La capa de interfaz se organiza alrededor de cuatro elementos principales:

Clase de entrada JavaFX. Inicializa la aplicación, crea la ventana principal y coordina el cambio entre pantallas.

Modelo usado por la interfaz. Mantiene la partida activa y expone métodos de alto nivel para mover, atacar, guardar, cargar o interactuar.

Controlador de la partida. Conecta eventos de teclado y botones con operaciones del modelo.

Cada vista representa una pantalla o panel concreto. Entre ellas están

`MainMenuView`, `StoryIntroView`, `CharacterSelectView`, `DescentIntroView`, `GameView`, `InventoryView`, `CombatLogView`, `FinalView` y `FinalSummaryView`.

9.3 Pantallas Principales

La experiencia visual se divide en pantallas diferenciadas:

Permite iniciar partida, cargar una partida existente o salir. También establece el tono visual del juego.

Presenta el estado de Valdris antes de que el jugador elija personaje. Sirve para contextualizar el conflicto y preparar la entrada al mundo.

Permite elegir entre Kael, Syra y Dorath. Muestra imagen, descripción, rol y estilo de juego de cada personaje.

Funciona como transición entre la elección de personaje y el inicio real de la partida. Refuerza la ambientación y avisa de que los movimientos y acciones están indicados y tienen teclas asociadas.

Es la pantalla principal durante la partida. Muestra la sala actual, el jugador, enemigos, cofres, objetos, puertas, turnos, inventario resumido y acciones.

Permite revisar objetos, equipar armas o armaduras y consumir pociones cuando la fase del turno lo permite.

Muestra acciones relevantes de movimiento, combate, objetos, puzzles y efectos de estado.

Muestra victoria o derrota, causa concreta y opciones posteriores.

Permite ver el resumen de partida antes de exportarlo a JSON.

9.4 Representación del Tablero

`GameView` representa la sala actual como una rejilla de celdas. Cada celda se

dibuja según su tipo y contenido.

La interfaz distingue visualmente:

- Suelo.
- Paredes.
- Puertas.
- Puertas ocultas descubiertas.
- Escaleras.
- Cofres.
- Palancas o activadores.
- Objetos del suelo.
- Jugador.
- Enemigos normales.
- Mini-bosses.
- Parásito y fases finales.
- Malachar.

Cada zona del mapa aplica un tono visual propio. Además, los pasillos de transición muestran un degradado entre zonas para reforzar el avance por el mundo.

9.5 Controles y Accesibilidad

La interfaz combina controles de ratón y teclado. Los botones permiten ejecutar acciones de forma explícita, mientras que las teclas asociadas aceleran la partida.

Las acciones principales están indicadas en pantalla:

- Movimiento.
- Recogida e interacción.
- Uso de objetos.
- Ataque.
- Esperar o terminar turno.
- Abrir inventario.
- Guardar partida.

El objetivo es que el jugador pueda entender qué acciones están disponibles sin tener que consultar documentación externa durante la partida.

9.6 Diseño Visual

El estilo visual mantiene una ambientación de fantasía oscura, pero prioriza la legibilidad. Las pantallas narrativas usan textos más envolventes y elementos ornamentales suaves, mientras que la pantalla de juego mantiene una estructura más funcional.

Las decisiones visuales principales son:

- Colores diferenciados por zona.
- Personajes con imágenes propias.
- Enemigos diferenciados por tipo y amenaza.
- Mini-bosses con representación más marcada.
- Cofres, armas, objetos y activadores reconocibles.
- Diálogos narrativos con estética integrada en el juego.
- Evitar ventanas que parezcan mensajes de error para eventos narrativos.

Esta estética busca que el juego sea más claro y atractivo sin perder la simplicidad de un tablero 2D por turnos.

9.7 Comunicación entre Modelo y Vista

`GameModelListener` permite que la vista se actualice cuando cambia el modelo.

Este patrón reduce el acoplamiento entre la lógica de partida y la interfaz.

Cuando el jugador realiza una acción:

- La vista o el controlador recibe el evento.
- El controlador invoca una operación del `GameModel`.
- El modelo delega en la lógica correspondiente.
- El estado de partida cambia.
- La vista se refresca para mostrar el nuevo estado.

Esta separación permite que el modelo sea más fácil de probar y que la interfaz se limite a representar el estado actual.

9.8 Bocetos y Recursos Visuales

La carpeta `imagenes/` contiene recursos visuales utilizados o tomados como base para la interfaz. Entre ellos destacan las imágenes de personajes y capturas o

referencias de diseño.

Estos recursos sirven como evidencia de trabajo previo de interfaz y como apoyo para explicar la evolución visual del proyecto. En la exportación final de la memoria se pueden incluir como anexos o como referencia dentro de esta sección.

10. Casos de Uso

10.1 Actores

El sistema tiene un actor principal:

Persona que inicia la aplicación, selecciona personaje, juega la partida, interactúa con el mapa, combate, guarda o carga y revisa el resultado final.

También existen actores internos no humanos:

Ejecuta reglas, controla turnos, aplica efectos, mueve enemigos y comprueba victoria o derrota.

Lee configuración JSON, guarda partidas, carga estados y exporta resúmenes.

10.2 Caso de Uso: Iniciar Partida Nueva

Actor principal: jugador.

Objetivo: comenzar una partida desde el estado inicial del mundo.

Flujo principal:

- El jugador abre el menú principal.
- Selecciona nueva partida.
- El sistema muestra la introducción narrativa.
- El jugador avanza a la selección de personaje.
- Elige Kael, Syra o Dorath.
- El sistema carga la configuración inicial desde JSON.
- El sistema genera el mundo y coloca al jugador en la sala inicial.
- Se muestra la pantalla de descenso.
- Comienza la partida en la pantalla de juego.

Resultado:

-
- La partida queda activa con personaje, mapa, sala inicial y turnos preparados.

10.3 Caso de Uso: Cargar Partida

Actor principal: jugador.

Objetivo: continuar una partida guardada previamente.

Flujo principal:

- El jugador pulsa cargar partida desde el menú principal.
- El sistema lee `partida_valdris.json`.
- Se reconstruye el dungeon y el estado guardado.
- Se restauran jugador, sala, inventario, enemigos y turnos.
- La interfaz muestra la partida cargada.

Flujo alternativo:

- Si no existe partida válida, el sistema informa del error y permanece en el menú o en la pantalla actual.

Resultado:

- El jugador continúa desde el último estado guardado válido.

10.4 Caso de Uso: Mover al Jugador

Actor principal: jugador.

Objetivo: desplazar al personaje por la sala actual.

Precondiciones:

- La partida está activa.
- El jugador está vivo.
- La fase permite movimiento.

Flujo principal:

- El jugador selecciona una celda de destino o usa controles de movimiento.
- El sistema calcula celdas alcanzables mediante BFS.
- Se valida que el destino sea transitable y esté dentro del alcance.
- El jugador se mueve a la nueva celda.
- Se registra la acción en el log.
- La vista actualiza la posición del jugador.

Resultado:

- El jugador queda situado en una celda válida y consume su acción de movimiento.

10.5 Caso de Uso: Cambiar de Sala

Actor principal: jugador.

Objetivo: pasar de una habitación a otra conectada.

Precondiciones:

- El jugador está sobre una puerta, escalera o acceso válido.
- La conexión existe en el grafo del dungeon.

Flujo principal:

- El jugador interactúa con el acceso.
- El sistema identifica la sala destino.
- Se actualiza la sala actual.
- El jugador aparece en la posición de entrada correspondiente.
- La fase del turno se reinicia.
- La vista muestra la nueva sala.

Resultado:

- El jugador entra en otra habitación y puede actuar desde la fase inicial.

10.6 Caso de Uso: Recoger o Usar Objeto

Actor principal: jugador.

Objetivo: obtener un objeto del mapa o usar un objeto del inventario.

Flujo de recogida:

- El jugador se coloca sobre una celda con objeto o junto a un cofre.
- Interactúa con el objeto o contenedor.
- El sistema valida que la fase permite la acción.
- El objeto pasa al inventario o se equipa si corresponde.
- Se muestra un mensaje descriptivo.
- Se actualiza la vista.

Flujo de uso:

- El jugador abre el inventario.

-
- Selecciona un objeto usable.
 - El sistema valida fase y efecto.
 - Se aplica el efecto del objeto.
 - Se registra la acción en el log.

Resultado:

- El inventario, equipo o estado del jugador queda actualizado.

10.7 Caso de Uso: Abrir Cofre Secreto

Actor principal: jugador.

Objetivo: obtener una recompensa relevante de una sala secreta.

Precondiciones:

- El jugador ha descubierto o accedido a la sala secreta.
- El cofre no ha sido abierto previamente.

Flujo principal:

- El jugador interactúa con el cofre.
- El sistema muestra las opciones de recompensa disponibles.
- El jugador elige una de las armas propuestas.
- El arma se añade al inventario.
- El cofre queda marcado como abierto.
- La interfaz muestra un mensaje narrativo con la recompensa obtenida.

Resultado:

- El jugador obtiene una recompensa única y el cofre no puede repetirse.

10.8 Caso de Uso: Combatir

Actor principal: jugador.

Objetivo: atacar a un enemigo de la sala actual.

Precondiciones:

- La partida está activa.
- El jugador está vivo.
- Hay un enemigo vivo en rango válido.
- La fase permite atacar.

Flujo principal:

- El jugador selecciona un enemigo objetivo.
- El sistema valida rango y línea de visión.
- Se calcula el daño con ataque, defensa, arma y efectos.
- Se aplica el daño al enemigo.
- Si el enemigo muere, se procesan recompensas o progreso.
- Se registra el resultado en el log.
- La vista actualiza vida y estado de unidades.

Resultado:

- El enemigo recibe daño o muere, y el jugador consume su acción de ataque.

10.9 Caso de Uso: Turno de Enemigos

Actor principal: sistema de juego.

Objetivo: resolver las acciones de los enemigos tras el turno del jugador.

Flujo principal:

- El jugador termina sus acciones o cede el turno.
- El sistema recorre los enemigos vivos de la sala.
- Cada enemigo decide acción mediante IA.
- Se ejecutan ataques, movimientos o habilidades.
- Se aplican efectos y daño al jugador cuando corresponda.
- Se comprueba si el jugador ha muerto.
- Se prepara el siguiente turno del jugador si la partida continúa.

Resultado:

- El estado de sala y jugador refleja las acciones enemigas resueltas.

10.10 Caso de Uso: Guardar Partida

Actor principal: jugador.

Objetivo: almacenar el estado actual para continuar más adelante.

Flujo principal:

- El jugador pulsa guardar partida.
- El sistema construye un `GameState`.
- `LectorJSON` serializa el estado con Gson.

-
- Se escribe `partida_valdriis.json`.
 - El sistema informa de que el guardado se ha completado.

Resultado:

- Existe un JSON actualizado con el estado de la partida.

10.11 Caso de Uso: Finalizar Partida

Actor principal: jugador.

Objetivo: Llegar a una condición de victoria o derrota y consultar el resultado.

Flujo de victoria:

- El jugador supera el combate final.
- El sistema muestra el desenlace narrativo.
- Se genera el resumen final.
- El jugador puede ver el resumen o volver al menú.
- Si lo desea, exporta `resumen_valdriis.json`.

Flujo de derrota:

- El jugador muere o se cumple una condición de derrota.
- El sistema identifica la causa concreta.
- Se muestra la pantalla final correspondiente.
- Se genera el resumen final.
- El jugador puede consultar o exportar el resumen.

Resultado:

- La partida termina y el resultado queda visible para el jugador.
-

11. Diagramas UML

11.1 Objetivo de los Diagramas

Los diagramas UML sirven para representar el diseño del sistema desde distintos puntos de vista. No sustituyen al código, pero permiten explicar visualmente las clases principales, los casos de uso, los flujos de interacción y los estados

relevantes de la partida.

El enunciado pide incluir varios tipos de diagramas. En esta memoria se referencian los existentes y se deja indicado qué diagramas deben añadirse antes de la entrega final.

11.2 Diagramas Disponibles

Actualmente el proyecto contiene la carpeta `diagramas/` con los siguientes ficheros:

- `diagramas/Estructura de clases 1.png`.
- `diagramas/Estructura de clases 2.png`.
- `diagramas/Fases de turno.png`.
- `diagramas/Fases de desarrollo.png`.
- `diagramas/diagrama_casos_uso.svg`.
- `diagramas/diagrama_casos_uso.puml`.
- `diagramas/diagrama_actividad_turno_jugador.svg`.
- `diagramas/diagrama_actividad_turno_jugador.puml`.
- `diagramas/diagrama_secuencia_ataque_jugador.svg`.
- `diagramas/diagrama_secuencia_ataque_jugador.puml`.
- `diagramas/diagrama_estados_partida.svg`.
- `diagramas/diagrama_estados_partida.puml`.

Los diagramas de estructura de clases cubren la parte principal del modelo orientado a objetos. Deben relacionarse con las secciones de modelo de dominio, herencia, estructuras de datos y arquitectura por capas.

El diagrama de fases de turno ayuda a explicar el comportamiento de la partida durante un turno. Es útil para justificar `TurnManager` y el flujo

`MOVEMENT -> PICKUP -> USE_ITEM -> ATTACK -> ENEMY_TURN`.

El diagrama de fases de desarrollo documenta la planificación y evolución del proyecto. Es útil como apoyo metodológico, aunque no sustituye a los diagramas UML obligatorios de comportamiento.

Los diagramas de casos de uso y actividad se incluyen en formato SVG para su consulta directa y en formato PlantUML para poder editarlos o regenerarlos si se

ajustan antes de la entrega.

Los diagramas de secuencia y estados completan la parte obligatoria de UML. Se mantienen también en SVG y PlantUML para conservar una versión visible y otra editable.

11.3 Diagrama de Casos de Uso

El diagrama de casos de uso se ha añadido en:

```
diagramas/diagrama_casos_uso.svg  
  
diagramas/diagrama_casos_uso.puml
```

Representa la interacción entre el jugador y el sistema.

Casos que deben aparecer:

- Iniciar partida nueva.
- Cargar partida.
- Elegir personaje.
- Mover jugador.
- Cambiar de sala.
- Recoger objeto.
- Abrir cofre.
- Usar objeto.
- Atacar enemigo.
- Guardar partida.
- Ver resumen final.
- Exportar resumen.

Actores recomendados:

- Jugador.
- Sistema de juego.
- Sistema de persistencia.

Este diagrama debe apoyarse en la sección 10 de esta memoria. No necesita mostrar todos los detalles internos, sino las funcionalidades principales desde el punto de vista del usuario.

11.4 Diagrama de Clases

El diagrama de clases debe mostrar las entidades principales del dominio y sus relaciones.

Elementos que deben aparecer:

- `Unit`, `Player`, `Enemy`, `MiniBossEnemy`, `ParasitoEnemy`.
- `Item`, `Weapon`, `Armor`, `Potion`, `Accessory`, `NarrativeItem`.
- `Dungeon`, `Room`, `Cell`.
- `Container`, `Chest`, `HiddenPassage`.
- `TurnManager`, `CombatManager`, `IAEnemy`, `ArbolDecisionIA`.
- `GameState`, `GameConfig`, `LectorJSON`, `DungeonConfigLoader`.
- `GameModel`, `GameController` y vistas principales.

Relaciones importantes:

- Herencia entre `Unit` y sus especializaciones.
- Herencia entre `Item` y sus especializaciones.
- Composición de `Dungeon` sobre salas.
- Composición de `Room` sobre `Cell[][]`.
- Uso de `Grafo<Room, String>` para conexiones.
- Dependencia de lógica sobre modelo.
- Dependencia de interfaz sobre modelo, lógica y persistencia.

Los ficheros `Estructura de clases 1.png` y `Estructura de clases 2.png` cubren esta parte de forma dividida para evitar un único diagrama demasiado grande.

11.5 Diagrama de Secuencia

El enunciado requiere al menos un diagrama de secuencia. Se ha añadido el ataque del jugador a un enemigo en:

```
diagramas/diagrama_secuencia_ataque_jugador.svg  
  
diagramas/diagrama_secuencia_ataque_jugador.puml
```

Participantes recomendados:

- `GameView`.
- `GameController`.
- `GameModel`.

-
- `TurnManager`.
 - `CombatManager`.
 - `Player`.
 - `Enemy`.
 - `CombatLogView`.

Flujo recomendado:

- El jugador selecciona un enemigo.
- `GameView` notifica la acción al controlador.
- `GameController` solicita atacar al `GameModel`.
- `GameModel` valida fase mediante `TurnManager`.
- `CombatManager` valida rango y línea de visión.
- `CombatManager` calcula daño y aplica efectos.
- `Enemy` actualiza su vida.
- `GameModel` registra el evento.
- La vista se refresca con el nuevo estado.

Este flujo es representativo porque atraviesa interfaz, controlador, modelo, reglas de turno, combate, entidades del dominio y log.

11.6 Diagrama de Estados

El enunciado pide al menos un diagrama de estados del juego. Se ha añadido el estado global de una partida en:

```
diagramas/diagrama_estados_partida.svg  
  
diagramas/diagrama_estados_partida.puml
```

Estados recomendados:

- Menú principal.
- Introducción narrativa.
- Selección de personaje.
- Descenso al núcleo.
- Partida activa.
- Turno del jugador.
- Turno de enemigos.
- Pausa o guardado.

-
- Victoria.
 - Derrota.
 - Resumen final.

Transiciones recomendadas:

- Nueva partida desde menú.
- Carga de partida desde menú.
- Elección de personaje.
- Inicio de combate o exploración.
- Fin de acciones del jugador.
- Fin del turno enemigo.
- Muerte del jugador.
- Derrota por turnos.
- Victoria tras jefe final.
- Consulta o exportación del resumen.

Este diagrama ayuda a explicar que la partida no es una pantalla única, sino una máquina de estados con transiciones controladas.

11.7 Diagrama de Actividad

El enunciado pide al menos un diagrama de actividad de una operación del jugador. Se ha añadido el turno completo del jugador en:

```
diagramas/diagrama_actividad_turno_jugador.svg  
diagramas/diagrama_actividad_turno_jugador.puml
```

Actividades recomendadas:

- Inicio de turno.
- Comprobar efectos activos.
- Elegir movimiento.
- Validar movimiento con BFS.
- Mover jugador.
- Interactuar o recoger objeto.
- Abrir inventario o usar objeto.
- Seleccionar ataque.
- Validar rango y visión.

-
- Resolver combate.
 - Comprobar victoria o derrota.
 - Pasar a turno de enemigos.

Este diagrama debe mostrar decisiones, no solo una lista lineal. Por ejemplo, debe distinguir si el jugador se mueve o no, si hay objeto disponible, si el ataque es válido o si la partida termina.

11.8 Estado de UML

Con los ficheros actuales, quedan cubiertos los diagramas obligatorios indicados por el enunciado:

- Diagrama de casos de uso.
- Diagrama de clases.
- Diagrama de secuencia.
- Diagrama de estados.
- Diagrama de actividad.

La carpeta **diagramas/** conserva los ficheros visuales y sus fuentes editables.

12. Contratos e Invariantes

12.1 Objetivo

Los contratos e invariantes definen condiciones que deben cumplirse para que el juego mantenga un estado coherente. Sirven para explicar qué presupone cada operación, qué garantiza al terminar y qué propiedades nunca deben romperse.

En Valdris son especialmente importantes porque el juego combina mapa, movimiento, combate, inventario, turnos, persistencia y eventos narrativos.

12.2 Invariantes del Jugador

El jugador debe cumplir siempre:

- La vida actual nunca puede ser menor que 0.
- La vida actual nunca debe superar la vida máxima salvo que una regla lo permita explícitamente.

-
- El jugador pertenece a una única sala actual.
 - La posición del jugador debe estar dentro de los límites de la sala.
 - La celda ocupada por el jugador debe ser transitable.
 - Solo puede haber un personaje jugable activo por partida.
 - El personaje elegido no cambia durante una partida.
 - El inventario no debe contener referencias nulas.
 - El equipo activo debe pertenecer al inventario o al estado válido del jugador.

Estas condiciones evitan estados imposibles, como un jugador fuera del tablero o un personaje con equipo inexistente.

12.3 Invariantes de Enemigos y Unidades

Las unidades del juego deben cumplir:

- Una unidad viva tiene vida mayor que 0.
- Una unidad muerta no debe actuar.
- La posición de una unidad debe estar dentro de la sala donde se encuentra.
- Dos unidades no deben ocupar la misma celda si la celda no lo permite.
- Los efectos activos deben tener duración válida.
- Un enemigo solo puede atacar si el jugador está en rango.
- Los ataques a distancia deben respetar la línea de visión cuando corresponde.

En enemigos especiales también se mantienen reglas propias. Por ejemplo, un guardián conserva una zona de referencia y el invocador debe respetar sus restricciones de invocación.

12.4 Invariantes del Mapa

El mapa global debe cumplir:

- Todo **Dungeon** contiene un grafo de salas.
- Cada sala tiene un identificador único.
- Cada sala contiene una matriz `cell[][]` no nula.
- Todas las filas y columnas de una sala son válidas para sus celdas.
- Las conexiones del grafo apuntan a salas existentes.
- Las puertas y escaleras deben corresponder con conexiones válidas.
- Las salas secretas solo son accesibles cuando se cumple su condición.
- El punto de no retorno debe respetar la dirección definida.

Estas condiciones aseguran que el jugador pueda progresar por el mundo sin caer en salas inexistentes o conexiones rotas.

12.5 Invariantes de Celdas

Cada celda debe mantener coherencia entre tipo, contenido y comportamiento.

Reglas principales:

- Una pared no debe ser transitable.
- Una puerta debe poder asociarse a una conexión.
- Una puerta oculta no debe actuar como acceso visible hasta ser descubierta.
- Una celda con cofre cerrado debe mostrar el cofre como interactuable.
- Una celda con objeto debe permitir recogida si la fase lo permite.
- Una celda con trigger debe distinguirse visualmente del suelo normal.
- Las coordenadas de una celda deben coincidir con su posición en la matriz.

Estas reglas conectan la representación visual con la lógica real de juego.

12.6 Invariantes de Turnos

El sistema de turnos debe cumplir:

- Solo existe una fase activa en cada momento.
- El jugador no puede repetir una acción ya consumida en el mismo turno.
- El movimiento se resuelve antes que recogida, uso de objeto y ataque.
- El turno enemigo solo comienza cuando terminan las acciones del jugador.
- Al cambiar de sala se reinicia la fase del jugador.
- Los efectos de estado se procesan en el momento establecido.
- Los contadores de turnos globales y de sala no pueden ser negativos.

Estas condiciones permiten que el juego sea predecible y que el jugador entienda por qué una acción está disponible o bloqueada.

12.7 Invariantes de Inventario y Objetos

El inventario debe cumplir:

- No debe contener objetos nulos.
- Un objeto consumible usado debe retirarse o marcarse como consumido.
- Un arma equipada debe ser de tipo arma.
- Una armadura equipada debe ser de tipo armadura.

-
- Un accesorio equipado debe ser de tipo accesorio.
 - Los objetos narrativos no deben tratarse como consumibles normales.
 - Las recompensas únicas de cofres secretos no deben duplicarse.

Estas reglas impiden errores como equipar una poción como arma o abrir varias veces un cofre único.

12.8 Invariantes de Persistencia

La persistencia debe mantener:

- Los identificadores de salas guardados deben existir en la configuración.
- Los identificadores de objetos guardados deben poder reconstruirse.
- La sala actual guardada debe existir.
- La posición guardada debe ser válida dentro de la sala actual.
- Los enemigos guardados deben pertenecer a una sala existente.
- Los cofres abiertos deben corresponder a contenedores reales.
- El JSON no debe depender de objetos JavaFX.

La carga debe rechazar o controlar estados incoherentes mediante excepciones de estado. Esto evita que un fichero corrupto produzca una partida a medias.

12.9 Contrato de Movimiento

Precondiciones:

- La partida está activa.
- El jugador está vivo.
- La fase permite movimiento.
- El destino está dentro de los límites de la sala.

Postcondiciones si el movimiento es válido:

- La posición del jugador cambia al destino.
- La acción de movimiento queda consumida.
- El log registra la acción.
- La vista puede refrescarse con una posición coherente.

Postcondiciones si el movimiento no es válido:

- La posición del jugador no cambia.
- La fase no avanza de forma incorrecta.
- Se informa del motivo mediante excepción o mensaje controlado.

12.10 Contrato de Ataque

Precondiciones:

- La partida está activa.
- El atacante está vivo.
- El objetivo está vivo.
- La fase permite atacar si el atacante es el jugador.
- El objetivo está dentro del rango permitido.
- La línea de visión es válida cuando el ataque lo requiere.

Postcondiciones:

- El objetivo recibe el daño calculado.
- La vida del objetivo no baja de 0.
- Si el objetivo muere, queda marcado como muerto.
- Se aplican recompensas o progreso si corresponde.
- La acción queda registrada en el log.
- La acción de ataque queda consumida si la realiza el jugador.

12.11 Contrato de Guardado

Precondiciones:

- Existe una partida activa o un estado final exportable.
- El estado mínimo del jugador y del mapa es válido.

Postcondiciones:

- Se genera un DTO serializable.
- El fichero JSON se escribe en la ruta correspondiente.
- El JSON no contiene referencias circulares.
- La partida puede reconstruirse desde el fichero si se trata de guardado.

Si ocurre un error de escritura o serialización, el sistema debe informar del fallo sin destruir el estado actual de la partida en memoria.

12.12 Contrato de Apertura de Cofre

Precondiciones:

- El cofre existe en la sala actual.
- El jugador está en una posición válida para interactuar.

-
- El cofre no ha sido abierto.
 - La fase permite interacción.

Postcondiciones:

- El jugador recibe la recompensa elegida o generada.
- El cofre queda marcado como abierto.
- La recompensa se muestra mediante mensaje narrativo.
- El log registra la apertura.
- El cofre no puede volver a entregar la misma recompensa.

12.13 Contrato de Final de Partida

Precondiciones:

- Se ha cumplido una condición de victoria o derrota.

Postcondiciones:

- El resultado queda fijado como victoria o derrota.
- La causa concreta queda registrada.
- El jugador no puede seguir actuando en la partida terminada.
- Se genera un resumen final visible.
- El resumen puede exportarse a JSON.

Este contrato evita que el juego siga procesando turnos después de haber terminado.

13. Estrategia de Pruebas

13.1 Objetivo de las Pruebas

La estrategia de pruebas busca comprobar que las reglas principales del juego funcionan correctamente y que los cambios realizados durante el desarrollo no rompen comportamiento ya implementado.

Las pruebas se centran especialmente en las capas no visuales, porque son las que contienen la lógica principal del proyecto:

- Modelo.
- Mapa.

-
- Ítems.
 - Unidades.
 - Combate.
 - Movimiento.
 - IA.
 - Turnos.
 - Persistencia.
 - Estructuras de datos.

La interfaz JavaFX se valida principalmente mediante pruebas manuales de uso, ya que depende de eventos visuales, navegación entre pantallas y experiencia de usuario.

13.2 Herramientas Utilizadas

El proyecto utiliza JUnit 5 para las pruebas automáticas. Los tests se encuentran en la carpeta `tests/`, organizada de forma paralela a `src/`.

También se utiliza Maven para ejecutar la suite completa de pruebas. La orden de validación principal es:

```
.\mvnw.cmd test
```

Durante el desarrollo, esta ejecución se ha usado para comprobar que los cambios en lógica, persistencia, balanceo o reglas de juego no provocaban regresiones.

13.3 Capas Cubiertas por Tests

Las pruebas automáticas cubren principalmente:

Se validan listas, colas, pilas, grafos y árboles incluidos en el proyecto base.

Estas estructuras son críticas porque sustituyen a las estructuras estándar de `java.util` en la lógica principal.

Se prueban unidades, efectos, ítems, celdas, salas y dungeon. El objetivo es garantizar que el estado interno se mantiene coherente.

Se comprueba BFS de movimiento dentro de sala, validación de celdas transitables, límites de movimiento y rutas globales entre habitaciones.

Se prueban ataques, defensa, rango, muerte de unidades, efectos de estado y restricciones asociadas al combate.

Se validan decisiones de enemigos, comportamiento por tipo y acciones especiales cuando corresponde.

Se comprueba el avance de fases, consumo de acciones, reinicio al cambiar de sala, contadores y condiciones de derrota.

Se validan guardado, carga, reconstrucción de estado, configuración inicial y resumen final.

13.4 Pruebas Manuales de Interfaz

La capa JavaFX no se prueba con JUnit de forma automática en el alcance final del proyecto. En su lugar, se han realizado pruebas manuales centradas en el flujo real de juego.

Escenarios revisados:

- Abrir el juego desde el menú principal.
- Iniciar partida nueva.
- Leer introducción narrativa.
- Seleccionar personaje.
- Entrar en la pantalla de juego.
- Mover al jugador por salas válidas.
- Cambiar de sala.
- Activar puertas ocultas.
- Abrir cofres.
- Elegir armas en cofres secretos.
- Usar inventario.
- Combatir enemigos normales.
- Combatir mini-bosses.
- Llegar al combate final.
- Ver victoria o derrota.
- Consultar resumen final.
- Exportar resumen.
- Guardar y cargar partida.

Estas pruebas permiten comprobar aspectos que no se aprecian en tests unitarios, como legibilidad, mensajes narrativos, visibilidad de entidades y coherencia visual entre pantallas.

13.5 Pruebas de Regresión

Cada vez que se ha corregido un error relevante, se ha intentado comprobar que el problema no volvía a aparecer.

Ejemplos de regresiones revisadas:

- El efecto **BLIND** debe caducar tras los turnos correspondientes.
- La causa de muerte debe mostrarse correctamente.
- El cambio de sala debe reiniciar la fase de turno.
- Los cofres deben mostrar la recompensa obtenida.
- Malachar y el Parásito deben verse correctamente.
- El resumen final debe poder consultarse antes de exportarse.

Estas pruebas son importantes porque el proyecto combina muchas reglas interconectadas. Un cambio visual o de persistencia puede afectar a turnos, estado de sala o mensajes finales si no se revisa con cuidado.

13.6 Pruebas sobre JSON

La persistencia se revisa mediante ficheros de ejemplo incluidos en el repositorio:

- `config/configuracion_inicial_valdris.json`.
- `partida_valdris.json`.
- `resumen_valdris.json`.

Las pruebas y revisiones sobre JSON buscan comprobar:

- Que la configuración inicial se carga correctamente.
- Que una partida guardada puede reconstruirse.
- Que el estado restaurado conserva sala, jugador, inventario y turnos.
- Que los cofres, enemigos y celdas modificadas mantienen su estado.
- Que el resumen final contiene información suficiente.

El uso de JSON legible facilita detectar errores de identificadores, campos ausentes o estados incoherentes.

13.7 Limitaciones de la Estrategia de Pruebas

La estrategia de pruebas tiene algunas limitaciones:

- No hay automatización completa de la interfaz JavaFX.
- No se simula de forma automática una partida completa de inicio a final.
- Algunas comprobaciones visuales dependen de revisión manual.
- El balanceo de dificultad se ha ajustado mediante pruebas de juego.

Aun así, la combinación de JUnit para lógica y revisión manual para interfaz es adecuada para el alcance del proyecto y permite justificar la estabilidad del sistema.

13.8 Evidencia de Pruebas

La evidencia de pruebas se encuentra en:

- La carpeta `tests/`.
- Las ejecuciones registradas en `TASKS.md`.
- Las notas de cambios y correcciones registradas en `COMMIT_LOG.md`.

Antes de la entrega final, conviene ejecutar de nuevo la suite completa y dejar registrado el resultado final en los documentos de seguimiento.

14. Uso de Inteligencia Artificial

14.1 Objetivo del Uso de IA

La inteligencia artificial se ha usado como herramienta de apoyo durante el desarrollo del proyecto. Su función ha sido ayudar a planificar, revisar, implementar, depurar y documentar el sistema.

El uso de IA no sustituye la responsabilidad del grupo. Las decisiones finales, la revisión del código, la aceptación de cambios y la validación del resultado corresponden a los integrantes del equipo.

14.2 Tareas Apoyadas por IA

La IA se ha utilizado en tareas como:

- Interpretar requisitos del enunciado.
- Proponer una planificación por bloques.
- Revisar estructura de paquetes y dependencias.
- Implementar clases siguiendo las reglas del proyecto.
- Detectar errores de compilación o comportamiento.
- Ajustar persistencia JSON.
- Mejorar textos narrativos.
- Revisar balanceo de enemigos, armas y personajes.
- Mejorar la estética JavaFX.
- Preparar secciones de documentación.
- Revisar que los cambios fueran coherentes con el diseño general.

También se ha usado para comparar el estado del proyecto con los requisitos de entrega y detectar elementos pendientes, como UML, memoria, bocetos, JSON o vídeo.

14.3 Control Humano

El grupo ha mantenido control sobre las decisiones importantes. Antes de aplicar cambios relevantes, se ha seguido un flujo de trabajo basado en revisión previa:

- Se plantea el problema o mejora.
- La IA propone una solución o bloque de trabajo.
- El grupo revisa la propuesta.
- El grupo autoriza o modifica el alcance.
- Se implementa el cambio.
- Se revisa el resultado.
- Se ejecutan pruebas cuando corresponde.
- El grupo decide si se hace commit.

Este proceso evita aplicar cambios sin entender su impacto y mantiene la dirección del proyecto bajo decisión humana.

14.4 Registro del Uso de IA

El registro principal del uso de IA se conserva en:

Este fichero actúa como diario de desarrollo asistido. Recoge sesiones, cambios realizados, decisiones tomadas, pruebas ejecutadas y estado del proyecto.

Además, **TASKS.md** contiene el seguimiento de tareas completadas y pendientes. Ambos documentos permiten reconstruir la evolución del trabajo y justificar qué partes fueron asistidas, revisadas o validadas.

El diario de IA recoge los elementos pedidos por el enunciado:

- Agente utilizado y configuración inicial.
- Instrucciones, restricciones y metodología aplicada.
- Prompts o peticiones relevantes del grupo.
- Resultados generados en cada bloque de trabajo.
- Modificaciones aceptadas, corregidas o descartadas.
- Problemas encontrados y reajustes realizados.
- Verificación aplicada tras cada cambio.
- Crítica del uso de IA y metodología final obtenida.

14.5 Criterios de Revisión

Los cambios generados con apoyo de IA se han revisado usando varios criterios:

- Que compilen correctamente.
- Que respeten las capas del proyecto.
- Que no usen estructuras prohibidas de `java.util`.
- Que no modifiquen `MisEstructurasDeDatos/`.
- Que mantengan encapsulamiento.
- Que sean coherentes con la narrativa y el diseño.
- Que no rompan tests existentes.
- Que se documenten en **TASKS.md** y **COMMIT_LOG.md** cuando corresponde.

Esta revisión es especialmente importante porque la IA puede proponer soluciones válidas técnicamente, pero no siempre alineadas con las restricciones concretas del enunciado.

14.6 Aportación Real al Proyecto

El uso de IA ha sido útil sobre todo para acelerar tareas de ingeniería y documentación, pero el proyecto sigue teniendo una estructura propia definida por el grupo.

La IA ha ayudado a:

- Mantener una visión global del proyecto.
- Detectar requisitos pendientes.
- Proponer mejoras visuales y narrativas.
- Revisar errores difíciles de localizar.
- Redactar documentación final de forma ordenada.

Sin embargo, los criterios de diseño, el equilibrio del juego, la aceptación de la experiencia jugable y la entrega final dependen del equipo.

14.7 Riesgos del Uso de IA

El uso de IA también tiene riesgos:

- Puede introducir código que no respete restricciones del enunciado.
- Puede asumir información incorrecta si no se revisan los ficheros reales.
- Puede generar documentación demasiado genérica.
- Puede proponer cambios demasiado amplios para un problema pequeño.
- Puede ocultar decisiones si no se explican correctamente.

Para reducir estos riesgos, se ha trabajado leyendo los ficheros del proyecto, limitando el alcance de cada bloque, ejecutando pruebas y documentando los cambios.

14.8 Conclusión sobre el Uso de IA

La IA se ha usado como herramienta de apoyo supervisada. Ha contribuido a la organización, depuración y documentación del proyecto, pero el grupo conserva la autoría, revisión y responsabilidad final sobre el resultado entregado.

El diario de IA y los documentos de seguimiento permiten demostrar que su uso ha sido trazable y que las decisiones importantes han sido revisadas antes de cerrarse.

15. Decisiones de Diseño

15.1 Grafo Global y Matriz Local

Una de las decisiones principales del proyecto ha sido separar el mapa en dos niveles:

- Grafo para representar conexiones entre salas.
- Matriz `cell[][]` para representar el interior de cada sala.

El grafo permite modelar un mundo con ramificaciones, salas secretas, pasillos y punto de no retorno. La matriz permite resolver movimiento, obstáculos, objetos, cofres y enemigos dentro de una habitación concreta.

Esta decisión evita mezclar dos problemas distintos. El recorrido global se razona como conexiones entre nodos, mientras que el movimiento táctico se resuelve sobre celdas.

15.2 Configuración Inicial desde JSON

El enunciado exige que la configuración inicial se cargue desde JSON. Para cumplirlo, se decidió crear una configuración determinista del mundo con salas, dimensiones, celdas, accesos, cofres, triggers, enemigos importantes y objetivo.

El fichero principal es:

```
config/configuracion_inicial_valdris.json
```

Esta decisión mejora la revisión del proyecto, porque el mapa base puede consultarse sin leer código Java. También facilita modificar el mundo sin tocar la lógica principal.

15.3 Aleatoriedad Controlada

Aunque la configuración inicial es determinista, se mantuvo aleatoriedad en elementos que aportan rejugabilidad:

- Posiciones válidas de enemigos normales.
- Drops de enemigos.

-
- Algunas recompensas no críticas.
 - Variaciones internas de puzzles.

La decisión evita que el juego pierda interés. El JSON define la estructura y las reglas, pero la partida concreta puede variar dentro de límites seguros.

15.4 Turnos por Fases

El sistema de turnos se diseñó con fases explícitas:

- Movimiento.
- Recogida o interacción.
- Uso de objeto.
- Ataque.
- Turno de enemigos.

Esta división hace que el juego sea más fácil de entender y de validar. Cada acción tiene un momento concreto y el sistema puede impedir repeticiones dentro del mismo turno.

También se decidió reiniciar la fase al entrar en una nueva sala. Esto evita que el jugador llegue a una habitación en una fase avanzada y quede en desventaja o en un estado confuso.

15.5 Cofres Secretos con Elección de Arma

Las salas secretas se usan como recompensa importante de exploración. Por eso se decidió que sus cofres ofrecieran armas relevantes en lugar de objetos completamente aleatorios.

El jugador puede elegir entre varias opciones. Esto tiene dos ventajas:

- Hace que la exploración secreta tenga más valor.
- Permite adaptar la recompensa al personaje o estilo de juego.

La zona 4 incluye un cofre con tres opciones porque corresponde a un punto más avanzado y cercano al tramo final.

15.6 Balanceo de Personajes y Armas

El balanceo se ajustó para que los primeros niveles no fueran excesivamente

difíciles. Se aumentó el daño base de personajes y armas y se redujeron algunas estadísticas de enemigos iniciales.

La intención fue mantener el reto sin bloquear al jugador demasiado pronto.

También se reforzó la importancia de obtener armas, ya que el daño del personaje y el equipo influyen de forma clara en la supervivencia.

15.7 JavaFX con Separación Modelo-Vista-Controlador

La interfaz se organizó con una separación práctica entre vista, controlador y modelo de interfaz:

- Las vistas dibujan pantallas y reciben eventos.
- `GameController` conecta acciones del usuario con operaciones del juego.
- `GameModel` encapsula el estado activo para la interfaz.

Esta decisión evita colocar reglas de combate, movimiento o persistencia dentro de componentes visuales. También facilita actualizar la vista cuando cambia el estado de la partida.

15.8 Mensajes Narrativos Integrados

Durante la fase final se decidió sustituir mensajes con apariencia genérica o de error por diálogos integrados en la estética del juego.

Esta decisión afecta a:

- Mensajes por personaje y zona.
- Apertura de cofres.
- Encuentros narrativos.
- Diálogo final con Malachar.
- Desenlace de victoria o derrota.

El objetivo es que la interfaz no rompa la inmersión cuando muestra información importante.

15.9 Diferenciación Visual de Zonas y Enemigos

Se mejoró la representación visual para que cada zona tenga identidad propia.

Las salas usan tonos diferenciados y los pasillos muestran transiciones entre

zonas.

También se diferenciaron visualmente enemigos, mini-bosses, Malachar y las fases del Parásito. Esta decisión mejora la lectura del tablero y evita que amenazas importantes parezcan enemigos normales.

15.10 Resumen Final Visible

Inicialmente el resumen final podía exportarse, pero no consultarse de forma cómoda dentro del juego. Se decidió añadir una pantalla de resumen visible antes de la exportación.

Con esto, el jugador puede revisar la partida sin abrir directamente el JSON.

La exportación queda como opción posterior, más útil para entrega, depuración o evidencia.

15.11 Uso de Excepciones Personalizadas

El proyecto usa excepciones específicas como `InvalidMoveException`, `InvalidAttackException` y `GameStateException`.

Esta decisión permite distinguir errores de movimiento, combate y estado del juego sin recurrir a excepciones genéricas. También hace más fácil mostrar mensajes controlados en la interfaz.

15.12 Documentación Continua

El desarrollo se ha acompañado de documentación continua mediante:

- `TASKS.md`.
- `COMMIT_LOG.md`.
- Guías de diseño en `docs/`.
- JSON de ejemplo.
- Diagramas.
- Esta memoria.

La decisión de documentar durante el desarrollo reduce el riesgo de reconstruir todo al final y permite justificar mejor la evolución del proyecto.

16. Gestión del Proyecto

16.1 Organización del Trabajo

El proyecto se ha organizado por bloques funcionales. Primero se desarrollaron las clases base del dominio y después se avanzó hacia mapa, lógica, persistencia, interfaz y documentación final.

Esta organización permitió construir el sistema desde capas inferiores hacia capas superiores:

- Enumerados, efectos e ítems.
- Unidades.
- Mapa.
- Lógica de juego.
- Persistencia.
- JavaFX.
- Depuración y mejoras finales.
- Documentación y entregables.

El orden es importante porque las capas superiores dependen de las inferiores.

Por ejemplo, la interfaz necesita que movimiento, combate, turnos y persistencia funcionen antes de poder ofrecer una experiencia completa.

16.2 Seguimiento con TASKS.md

TASKS.md se ha usado como documento de seguimiento. Su función es registrar:

- Bloques terminados.
- Tareas pendientes.
- Correcciones aplicadas.
- Estado de pruebas.
- Cambios relevantes de alcance.
- Puntos pendientes de entrega.

Este fichero permite conocer rápidamente el estado del proyecto sin revisar todo el historial de commits o el código fuente.

16.3 Diario de Desarrollo con COMMIT_LOG.md

`COMMIT_LOG.md` se ha usado como diario de progreso. Recoge sesiones de trabajo, decisiones, pruebas realizadas y cambios importantes.

También funciona como evidencia del uso de IA, porque documenta cómo se ha trabajado con asistencia y qué cambios se han aceptado finalmente.

La combinación de `TASKS.md` y `COMMIT_LOG.md` proporciona trazabilidad:

- `TASKS.md` muestra qué falta y qué está cerrado.
- `COMMIT_LOG.md` explica cómo se ha llegado a ese estado.

16.4 Flujo de Cambios

El flujo habitual de trabajo ha sido:

- Detectar requisito, error o mejora.
- Revisar ficheros relevantes.
- Proponer una solución.
- Validar el enfoque antes de implementar cambios grandes.
- Modificar el código o la documentación.
- Ejecutar pruebas cuando corresponde.
- Actualizar documentos de seguimiento.
- Revisar el resultado.
- Hacer commit si el grupo lo autoriza.

Este flujo evita cambios grandes sin validación previa y facilita mantener el control del proyecto en una fase de depuración final.

16.5 Control de Versiones

El proyecto usa Git para controlar versiones. Los commits se realizan tras bloques de trabajo cerrados, no después de cada edición menor.

Esta decisión permite que cada commit represente una mejora o corrección coherente:

- Implementación de una funcionalidad.
- Corrección de errores.
- Mejora visual.
- Ajuste de balanceo.
- Actualización documental.

El grupo revisa los cambios antes de autorizar commits finales.

16.6 Gestión de Riesgos

Durante el desarrollo se identificaron varios riesgos:

- Que la interfaz JavaFX llegara tarde respecto a la lógica.
- Que la configuración JSON eliminara toda aleatoriedad.
- Que los primeros niveles fueran demasiado difíciles.
- Que los mensajes visuales rompieran la estética del juego.
- Que los UML y la memoria quedaran incompletos.
- Que algún cambio de persistencia rompiera la carga de partidas.

Estos riesgos se trataron mediante trabajo por bloques, pruebas frecuentes y revisión del alcance antes de implementar cambios grandes.

16.7 Estado Actual de Entrega

En el estado actual, el proyecto tiene implementado el núcleo jugable completo:

- Juego por turnos.
- Mapa por grafo y salas por matriz.
- Configuración inicial JSON.
- Guardado y carga.
- Combate.
- IA enemiga.
- Cofres y armas.
- Puzzles y salas secretas.
- Interfaz JavaFX.
- Pantallas narrativas.
- Resumen final visible y exportable.

Los puntos pendientes principales se concentran en documentación final, diagramas UML restantes, revisión final de memoria y empaquetado de entrega.

16.8 Cierre del Proyecto

El cierre del proyecto debe seguir estos pasos:

- Terminar los diagramas UML pendientes.
- Actualizar esta memoria con los nombres definitivos de los diagramas.

-
- Revisar ortografía, formato y coherencia final de esta memoria.
 - Ejecutar la suite de tests completa.
 - Registrar el resultado final en `TASKS.md` y `COMMIT_LOG.md`.
 - Exportar la memoria a PDF.
 - Preparar el ZIP o repositorio final.
 - Adjuntar vídeo explicativo externo.

Este cierre asegura que el código, la documentación y los entregables del enunciado quedan alineados.

17. Crítica del Proyecto

17.1 Alcance de la Crítica

El proyecto alcanza el objetivo principal: un juego completo por turnos con mapa, combate, IA, objetos, persistencia, interfaz JavaFX, narrativa y resumen final. Aun así, como en cualquier proyecto de alcance académico, existen limitaciones y mejoras posibles.

Esta sección funciona como la **crítica del proyecto** exigida por el enunciado. Recoge fallos, limitaciones y líneas de mejora sin ocultar que el núcleo jugable está completo.

Estas limitaciones no impiden jugar ni entregar el sistema, pero ayudan a identificar cómo podría evolucionar Valdris si se ampliara el desarrollo.

17.2 Automatización de Pruebas JavaFX

La interfaz se ha validado mediante pruebas manuales. Esto permite revisar flujo, estética y usabilidad, pero no sustituye a una automatización completa de eventos visuales.

Una mejora futura sería incorporar pruebas específicas de JavaFX para simular:

- Pulsaciones de botones.
- Selección de personaje.
- Movimiento por teclado.

-
- Apertura de inventario.
 - Diálogos.
 - Pantalla final.

Esto reduciría el riesgo de que un cambio visual rompa navegación o controles.

17.3 Deshacer Acciones con Pila

El proyecto dispone de `Pila<T>` como estructura de datos, pero no implementa una función completa de deshacer acciones.

Una ampliación posible sería guardar acciones del jugador en una pila:

- Movimiento anterior.
- Objeto recogido.
- Cambio de equipo.
- Acción de turno.

Con ello se podría restaurar el estado previo en situaciones controladas. Esta mejora requeriría definir cuidadosamente qué acciones pueden deshacerse y cuáles no, especialmente si ya han actuado los enemigos.

17.4 Dijkstra con Costes Variables

El mapa global usa BFS porque las conexiones entre habitaciones no tienen pesos variables en la lógica principal.

Una mejora futura sería introducir costes distintos:

- Pasillos peligrosos.
- Zonas malditas.
- Rutas bloqueadas temporalmente.
- Salas con penalización de turnos.

En ese caso tendría sentido usar Dijkstra sobre el grafo para calcular rutas de menor coste, no solo menor número de conexiones.

17.5 Más Variedad de Puzzles

El juego incluye puzzles y activadores, pero podrían ampliarse con más tipos de desafíos:

-
- Secuencias de palancas.
 - Patrones de presión en el suelo.
 - Puzzles de visión o luz.
 - Salas con turnos limitados más complejos.
 - Decisiones narrativas con consecuencias.

Esta mejora aumentaría la variedad de exploración sin cambiar el núcleo del sistema de turnos.

17.6 Animaciones y Sonido

La interfaz visual cumple el objetivo de representar el juego, diferenciar zonas y mostrar entidades reconocibles. Una mejora futura sería añadir animaciones y sonido:

- Animación de ataque.
- Transición entre salas.
- Efectos visuales de estado.
- Sonidos de cofre, daño o victoria.
- Música ambiental por zona.

Estas mejoras aportarían inmersión, aunque también aumentarían la complejidad de recursos y pruebas.

17.7 Balanceo Avanzado

El balanceo se ajustó durante la fase final para que los primeros niveles fueran más jugables. Aun así, el equilibrio de un juego siempre puede refinarse con más partidas de prueba.

Mejoras posibles:

- Ajustar curvas de daño por zona.
- Revisar drops por tipo de enemigo.
- Medir duración media de partida.
- Ajustar vida y defensa de mini-bosses.
- Crear dificultad seleccionable.

Estas mejoras permitirían adaptar el reto a distintos tipos de jugador.

17.8 Más Contenido Narrativo

La narrativa actual introduce el mundo, personajes, zonas y desenlace final. Una posible ampliación sería añadir más diálogos intermedios:

- Comentarios por personaje al encontrar armas.
- Reacciones ante mini-bosses.
- Recuerdos de Malachar.
- Textos especiales en salas secretas.
- Finales con pequeños matices según objetos encontrados.

Esto reforzaría el tono de fantasía oscura y daría más peso a la elección de personaje.

17.9 Editor o Validador de Configuración JSON

La configuración inicial en JSON facilita revisar y modificar el mapa, pero un error de identificadores puede romper la carga.

Una mejora futura sería crear un validador específico que comprobara:

- Salas duplicadas.
- Conexiones inexistentes.
- Puertas sin sala destino.
- Cofres sin recompensa.
- Triggers sin objetivo.
- Celdas fuera de rango.

También podría desarrollarse un pequeño editor de mapas para modificar salas de forma visual.

17.10 Empaquetado Ejecutable

El proyecto se ejecuta como aplicación JavaFX dentro del entorno de desarrollo.

Como mejora futura, se podría preparar un empaquetado más cómodo:

- Ejecutable con dependencias incluidas.
- Script de arranque.
- Distribución por sistema operativo.
- Carpeta de recursos organizada para entrega externa.

Esto facilitaría probar el juego fuera del entorno de desarrollo.

18. Conclusiones

18.1 Resultado General

Valdris: El Núcleo Profundo ha evolucionado desde una propuesta de juego por turnos hasta una aplicación completa en Java 21 y JavaFX. El resultado final incluye exploración, combate, IA, objetos, armas, puzzles, persistencia, narrativa, interfaz gráfica y resumen de partida.

El proyecto cumple el objetivo principal de las asignaturas: aplicar diseño orientado a objetos, estructuras de datos propias, algoritmos, excepciones, persistencia, pruebas y documentación en un sistema funcional.

18.2 Cumplimiento del Enunciado

El proyecto cubre los elementos más relevantes del enunciado:

- Código fuente organizado por capas.
- Uso de Java 21.
- Interfaz JavaFX funcional.
- Estructuras de datos propias.
- Grafo para el mapa.
- BFS para movimiento y caminos.
- Árbol de decisión para IA.
- Persistencia JSON.
- Excepciones personalizadas.
- Tests JUnit.
- Memoria y documento de diseño.
- Diario de uso de IA.
- Bocetos y recursos visuales.
- Diagramas UML en proceso de cierre final.
- Vídeo explicativo preparado como entregable externo.

Los puntos pendientes antes de entregar son de cierre documental: completar los UML restantes, actualizar esta memoria con sus nombres definitivos, exportarla a PDF y registrar la validación final.

18.3 Valor Técnico

Desde el punto de vista técnico, el proyecto destaca por combinar varias piezas:

- Un modelo de dominio suficientemente amplio.
- Una separación clara entre modelo, lógica, persistencia e interfaz.
- Un mapa representado con grafo y salas con matriz.
- Movimiento con BFS.
- Combate con efectos y modificadores.
- IA diferenciada por tipo de enemigo.
- Carga inicial desde JSON.
- Guardado, carga y resumen final.

Esta combinación demuestra que las estructuras y algoritmos no aparecen de forma aislada, sino integrados en un juego completo.

18.4 Valor de Diseño

El diseño del juego busca equilibrar requisitos académicos y experiencia de usuario. Algunas decisiones se tomaron para cumplir el enunciado, como JSON, grafos, BFS o estructuras propias. Otras se tomaron para mejorar la experiencia, como las pantallas narrativas, la diferenciación visual de zonas, los cofres con elección de armas y el resumen visible.

El resultado es un sistema más completo que una demostración técnica. El jugador puede iniciar una partida, entender el contexto, elegir personaje, progresar, mejorar equipo, enfrentarse a enemigos y llegar a un desenlace.

18.5 Aprendizaje del Proyecto

El desarrollo ha permitido trabajar aspectos importantes de ingeniería:

- Planificación por bloques.
- Control de dependencias entre capas.
- Diseño orientado a objetos.
- Persistencia de estados complejos.
- Depuración de errores de juego.
- Revisión de balanceo.
- Documentación continua.

-
- Integración de interfaz gráfica.
 - Uso supervisado de IA.

También ha mostrado la importancia de revisar requisitos al final del proyecto.

Aunque el juego estuviera casi completo, todavía quedaban entregables documentales imprescindibles como UML, memoria, referencias a bocetos y justificación formal de estructuras y costes.

18.6 Cierre

Valdris queda como un proyecto completo y defendible. Su código implementa una partida funcional y su documentación explica las decisiones principales, estructuras, algoritmos, pruebas, persistencia y uso de IA.

La última fase debe centrarse en pulir la entrega, no en cambiar el núcleo del juego: cerrar diagramas, revisar ortografía, ejecutar tests finales, exportar la memoria y preparar el paquete definitivo.

19. Anexos

19.1 Archivos Principales del Proyecto

Archivos de referencia del repositorio:

- `PROJECT_SPEC.md`.
- `INTRUCCIONES_GENERALES.md`.
- `TASKS.md`.
- `COMMIT_LOG.md`.
- `MEMORIA_DISENO_VALDRIS.md`.

Estos ficheros permiten revisar especificación, requisitos, progreso, seguimiento y memoria final.

19.2 Código Fuente

El código principal se encuentra en:

```
src/Valdris/
```

Las estructuras de datos propias se encuentran en:

```
src/MisEstructurasDeDatos/
```

La carpeta `MisEstructurasDeDatos/` contiene listas, listas circulares, colas, pilas, grafos y árboles reutilizados en el proyecto.

19.3 Pruebas

Las pruebas unitarias se encuentran en:

```
tests/
```

La suite se ejecuta mediante Maven:

```
.\mvnw.cmd test
```

El resultado final de la última ejecución debe registrarse en `TASKS.md` y `COMMIT_LOG.md` antes de cerrar la entrega.

19.4 JSON de Ejemplo

Los ficheros JSON relevantes son:

```
config/configuracion_inicial_valdris.json  
  
partida_valdris.json  
  
resumen_valdris.json
```

Cada uno cumple una función distinta:

- Configuración inicial del mundo.
- Estado guardado de partida.
- Resumen final exportable.

Estos ficheros sirven como evidencia de entrada, persistencia y salida en JSON.

19.5 Diagramas

Los diagramas actuales se encuentran en:

diagramas/

Ficheros existentes:

- diagramas/Estructura de clases 1.png.
- diagramas/Estructura de clases 2.png.
- diagramas/Fases de turno.png.
- diagramas/Fases de desarrollo.png.
- diagramas/diagrama_casos_uso.svg.
- diagramas/diagrama_casos_uso.puml.
- diagramas/diagrama_actividad_turno_jugador.svg.
- diagramas/diagrama_actividad_turno_jugador.puml.
- diagramas/diagrama_secuencia_ataque_jugador.svg.
- diagramas/diagrama_secuencia_ataque_jugador.puml.
- diagramas/diagrama_estados_partida.svg.
- diagramas/diagrama_estados_partida.puml.

Con esta lista quedan referenciados los UML obligatorios: casos de uso, clases, secuencia, estados y actividad. Antes de exportar la memoria a PDF solo conviene revisar visualmente que las imágenes se insertan correctamente en el documento final.

19.6 Imágenes y Bocetos

Los recursos visuales y bocetos se encuentran en:

imagenes/

Esta carpeta incluye imágenes usadas como apoyo visual, referencias de interfaz y recursos de personajes. Sirve como evidencia de diseño previo y evolución visual del juego.

19.7 Documentación Interna

La documentación de diseño y guías internas se encuentra principalmente en:

docs/

Estas guías han servido como base para:

-
- Narrativa.
 - Personajes.
 - Balanceo.
 - Armas.
 - Diseño visual.
 - Estructura del juego.
 - Requisitos técnicos.

La memoria consolida esa información y la adapta al estado final implementado.

19.8 Vídeo Explicativo

El vídeo explicativo es un entregable externo al repositorio. Debe mostrar:

- Presentación del proyecto.
- Participación de los miembros del grupo.
- Funcionamiento general del juego.
- Interfaz JavaFX.
- Movimiento y combate.
- Guardado o carga.
- Elementos destacados de diseño.
- Conclusión de la entrega.

Aunque no se incluya directamente en el repositorio, debe adjuntarse según las instrucciones de entrega.

19.9 Revisión Final Recomendada

Antes de entregar, se recomienda comprobar:

- Que todos los UML obligatorios están incluidos.
- Que esta memoria está actualizada con los diagramas definitivos.
- Que la memoria se ha exportado correctamente a PDF.
- Que no hay errores ortográficos evidentes.
- Que los JSON de ejemplo existen y son legibles.
- Que `TASKS.md` y `COMMIT_LOG.md` reflejan el estado final.
- Que la suite de tests pasa.
- Que el vídeo está preparado.
- Que el ZIP o repositorio contiene todos los ficheros necesarios.

Esta lista sirve como control final para evitar entregar el proyecto con un

requisito documental pendiente.